On the Limits of Online Lookahead for Fragmentation-Aware MIG GPU Scheduling

Saurabh Kumar
Department of Computer Science and Engineering
Indian Institute of Technology (BHU) Varanasi, India
saurabh.kumar.cse@iitbhu.ac.in

***Abstract*—Multi-tenant GPU clusters increasingly rely on NVIDIA Multi-Instance GPU (MIG) hardware partitioning to enforce compute and memory isolation across co-located deep learning workloads. However, rigid hardware partitioning rules across dual resource dimensions (7 compute slices and 8 memory slices on NVIDIA A100-80GB GPUs) induce severe spatial fragmentation. Recent state-of-the-art heuristics, notably the Minimum Fragmentation Increment (MFI), make greedy placement decisions to minimize future blocked configurations. In this work, we investigate whether online bounded lookahead via Monte Carlo rollouts can surpass myopic heuristics by anticipating downstream arrival waves. Through an extensive empirical evaluation across a 20-GPU cluster evaluating 160 matched workload traces (1,600 full simulation runs) under progressive streaming and concurrent burst regimes, we demonstrate that online lookahead yields no statistically meaningful acceptance gains over greedy MFI. Under progressive load, lookahead matches MFI identically (0 wins, 0 losses, 80 ties; paired delta 0.00 percentage points (pp), 95% bootstrap CI [0.00, 0.00]); under burst load, lookahead achieves 73.00% vs. 72.88% (+0.12 pp, 95% CI [−0.05, +0.34] pp, crossing zero). Simultaneously, lookahead imposes an intolerable 50×–70× decision latency overhead (24–40 ms vs. 0.5–0.7 ms for MFI), even when bounded by GPU-diverse candidate pruning. Furthermore, we resolve a key architectural ambiguity in MFI's mathematical definition, demonstrating that free-anchor filtering reproduces published Figure 3 scores ($F = 8, 16$) whereas literal all-starts enumeration produces inflated scores ($F = 14, 23$) and shifts 74.6% of placement decisions. Finally, we show that under zero-gap burst arrivals, load-spreading heuristics (Worst-Fit Best-Index) outperform greedy MFI by +3.63 pp (76.51% vs. 72.88%, 95% CI [+2.47, +4.79] pp), proving that greedy spatial packing causes catastrophic head-of-line blocking under concurrent arrival waves. We conclude that online lookahead has negative operational ROI and recommend greedy MFI for streaming workloads and load-spreading for bursty regimes.**

***Keywords*—GPU Scheduling, Multi-Instance GPU (MIG), Fragmentation, Lookahead Search, Cloud Computing, Empirical Systems Research.**

Introduction

Accelerating deep learning training and inference workloads has elevated Graphics Processing Units (GPUs) to the status of primary compute engines in enterprise datacenters and public cloud environments [1, 2]. Modern datacenter GPUs, such as the NVIDIA A100-80GB Tensor Core GPU, offer unprecedented levels of hardware parallelism, comprising 108 Streaming Multiprocessors (SMs) grouped into 7 Graphics Processing Clusters (GPCs) and 80 GB of high-bandwidth memory (HBM2e) served by 8 physical memory controllers.

However, individual enterprise workloads—ranging from small inference microservices and parameter-tuning tasks to interactive model evaluation—frequently require only a small fraction of an A100′s aggregate compute capacity [2, 3]. Traditional software-level sharing mechanisms, such as NVIDIA CUDA Multi-Process Service (MPS) or time-slicing, suffer from "noisy neighbor" interference, lack fault isolation, and cannot provide quality-of-service (QoS) guarantees.

To address these limitations, NVIDIA introduced Multi-Instance GPU (MIG) architecture [2]. MIG partitions an A100 GPU at the hardware level into up to seven fully isolated GPU instances. Each instance is allocated dedicated compute slices (GPCs), dedicated memory slices (memory controllers and DRAM banks), private crossbar paths, and isolated Level-2 cache partitions. This hardware isolation guarantees that memory bandwidth contention or hardware faults in one instance cannot degrade another instance running on the same physical chip.

Despite these isolation benefits, MIG imposes rigid spatial constraints [2, 3]. Instances cannot be instantiated at arbitrary physical offsets; rather, each instance profile requires a specific contiguous sequence of compute and memory slices beginning at vendor-dictated legal start indices. When multi-tenant workloads arrive dynamically online, assigning

a profile to an unfortunate start index can split remaining unallocated resources into disjoint slivers, causing severe spatial fragmentation. Under high fragmentation, incoming jobs requesting large profiles are rejected even when the GPU possesses ample total free compute and memory [3].

To combat this challenge, Zambianco et al. [3] recently introduced the Minimum Fragmentation Increment (MFI) heuristic. MFI evaluates every legal placement for an incoming job and selects the assignment that minimizes the increase in a cluster-wide fragmentation score. The fragmentation score quantifies the aggregate memory capacity of all valid MIG profile configurations that become blocked by the placement.

While MFI represents a notable advancement over classical heuristics such as First-Fit (FF), Best-Fit (BF), and Worst-Fit (WF), it operates in a strictly myopic, greedy fashion. A compelling research question naturally emerges: **Can an online scheduler surpass greedy MFI by incorporating bounded predictive lookahead?** Specifically, if an online scheduler evaluates candidate placements by simulating hypothetical future arrivals via Monte Carlo rollouts, can it avoid greedy traps, preserve contiguous physical capacity for anticipated large jobs, and achieve higher workload acceptance rates?

In this paper, we systematically investigate the viability, performance bounds, and computational limits of online lookahead for MIG scheduling. Over a 14-milestone investigative program, we design an executable dual-resource MIG simulation platform, formally reconstruct and validate baseline conformance, develop adaptive and fixed-budget lookahead algorithms with GPU-diverse candidate pruning, and conduct a large-scale empirical study across 20 simulated GPUs, 160 matched workload traces, and 1,600 simulation runs.

Our empirical findings reveal decisive negative and positive insights:

1. **Neutrality of Online Lookahead at Scale:** Under progressive streaming load, lookahead produces zero statistically significant gain over greedy MFI (80 ties in 80 paired traces; paired delta 0.00 pp). Under burst arrivals, lookahead achieves a statistically indistinguishable +0.12 pp delta (95% bootstrap CI [−0.05, +0.34] pp).
2. **Prohibitive Computational Overhead:** Lookahead incurs a 50×–70× latency penalty (24–40 ms vs. 0.5–0.7 ms for MFI), rendering it unsuitable for online cloud orchestrators.
3. **Mathematical Score Resolution:** We resolve an ambiguity in published MFI scores, proving that filtering out occupied anchor indices reproduces published values ($F = 8, 16$) while literal all-starts enumeration shifts 74.6% of placement choices.
4. **Dominance of Spreading Under Burst Load:** Under zero-gap burst arrivals, Worst-Fit Best-Index (`paper_wf_bi`) outperforms MFI by +3.63 pp (76.51% vs. 72.88%, 95% CI [+2.47, +4.79] pp), proving that greedy spatial packing is fundamentally counter-productive during concurrent arrival waves.

System Model & Dual-Resource Architecture

Physical Hardware Abstraction

We model a homogeneous cluster of G NVIDIA A100-80GB GPUs. At the hardware level, an A100-80GB GPU consists of two distinct physical resource dimensions:

- **Compute Resources:** $C = 7$ physical compute slices (GPCs).
- **Memory Resources:** $M = 8$ physical memory slices (each representing approximately 10 GB of HBM2e memory and associated memory controller paths).

Let the physical allocation state of GPU $g \in \{1, ..., G\}$ be represented by a dual occupancy bitvector $\left(u_g^{\text{comp}}, u_g^{\text{mem}}\right)$, where:

$$u_g^{\text{comp}} \in \{0,1\}^C, \quad u_g^{\text{mem}} \in \{0,1\}^M$$

Here, $u_g^{\text{comp}}[i] = 1$ indicates that compute slice $i \in \{0, ..., 6\}$ is occupied, and $u_g^{\text{mem}}[j] = 1$ indicates that memory slice $j \in \{0, ..., 7\}$ is occupied. The aggregate available compute and memory on GPU g are given by:

$$C_g^{\text{free}} = C - \sum_{i=0}^{C-1} u_g^{\text{comp}}[i], \quad M_g^{\text{free}} = M - \sum_{j=0}^{M-1} u_g^{\text{mem}}[j]$$

Legal MIG Profiles and Alignment Constraints

NVIDIA defines six legal instance profiles for the A100-80GB GPU [2, 3], denoted by the set $\mathcal{P} = \{p_1, ..., p_6\}$. Each profile p requires a fixed number of contiguous compute slices c_p and memory slices m_p, and may only be instantiated at a vendor-defined set of legal start indices $\mathcal{S}_p \subset \{0, ..., C-1\}$.

Profile (p)	Compute (c_p)	Memory (m_p)	Paper Slices	Legal Starts ($\mathcal{S}_p$)
`7g.80gb`	7	8	7	{0}
`4g.40gb`	4	4	4	{0}
`3g.40gb`	3	4	4	{0, 4}
`2g.20gb`	2	2	2	{0, 2, 4}
`1g.20gb`	1	2	2	{0, 2, 4, 6}
`1g.10gb`	1	1	1	{0, 1, 2, 3, 4, 5, 6}

Table 1: A100-80GB Legal MIG Profiles and Hardware Parameters [2, 3].

Table 1 details the physical footprint of each legal profile. A critical architectural nuance is the **dual-resource mismatch** present in asymmetric profiles:

- Profile `3g.40gb` consumes 3 compute slices but 4 memory slices ($c_p = 3, m_p = 4$). Instantiating `3g.40gb` at compute start index 4 occupies compute slices $\{4, 5, 6\}$ and memory slices $\{4, 5, 6, 7\}$.

- Profile `1g.20gb` consumes 1 compute slice but 2 memory slices ($c_p = 1, m_p = 2$). Instantiating it at compute start 2 occupies compute slice 2 and memory slices $\{2, 3\}$.

Feasibility Definition

A candidate placement of profile p at start index $s \in \mathcal{S}_p$ on GPU g is physically feasible if and only if all required contiguous compute and memory slices are simultaneously unoccupied:

$$\sum_{i=s}^{s+c_p-1} u_g^{\text{comp}}[i] = 0 \quad \wedge \quad \sum_{j=s}^{s+m_p-1} u_g^{\text{mem}}[j] = 0$$

Upon placement, the state of GPU g transitions to $\left(u_g^{\text{comp}} + p_s, u_g^{\text{mem}} + p_s\right)$, setting the corresponding slice ranges to 1. When the job completes, the slices are set back to 0.

Turkkan Best-Index Preference Ordering

When multiple start indices are feasible for a given profile on a GPU, simple heuristics such as First-Fit choose the lowest index. However, Turkkan et al. [4] observed that arbitrary index choices fragment the GPU, blocking larger profiles. By reverse-engineering hardware partition tables via `nvidia-smi`, Turkkan et al. derived an empirical "Best Index" (BI) preference ordering designed to preserve critical anchor locations:

- `3g.40gb`: $(4, 0)$
- `2g.20gb`: $(4, 0, 2)$
- `1g.20gb`: $(6, 4, 0, 2)$
- `1g.10gb`: $(6, 4, 5, 0, 1, 2, 3)$

Placing a `1g.10gb` instance at start index 6, for instance, leaves start index 0 open for potential `4g.40gb` or `7g.80gb` allocations, mitigating unnecessary fragmentation.

Fragmentation Increment & Baseline Conformance

Mathematical Definition of MFI

Zambianco et al. [3] formalized fragmentation by quantifying the penalty of blocked configurations. For any GPU g, let $\mathcal{P}_g^{\text{cand}}$ denote the candidate subset of profiles that could theoretically be supported by the GPU's remaining aggregate resources:

$$\mathcal{P}_g^{\text{cand}} = \left\{ p \in \mathcal{P} \;\middle|\; C_g^{\text{free}} \geq c_p \wedge M_g^{\text{free}} \geq m_p \right\}$$

Profiles failing this aggregate gate are excluded from the penalty, preventing fully saturated GPUs from being penalized for unaccommodated workloads.

For each candidate profile $p \in \mathcal{P}_g^{\text{cand}}$, let $\mathcal{B}(g, p)$ denote the set of legal start indices that are **blocked** due to physical slice occupancy:

$$F(g) = \sum_{p \in \mathcal{P}_g^{\text{cand}}} m_p \cdot |\mathcal{B}(g, p)|$$

The cluster fragmentation score is $F = \sum_{g=1}^{G} F(g)$. When a new job arrives requesting profile p_j, the Minimum Fragmentation Increment (MFI) heuristic evaluates every feasible placement $a = (g, s)$ and selects the one that minimizes the resulting change in fragmentation:

$$a^* = \text{argmin}_{a \in \mathcal{A}(j)} [F(g + a) - F(g)]$$

The Figure 3 Conformance Resolution

During our audit of MFI [3], we encountered a foundational ambiguity. In Section V and Figure 3 of [3], the authors present two reconstructed GPU states:

- **GPU 1:** Occupied by two `1g.10gb` instances at indices 2 and 3 ($C^{\text{free}} = \frac{5}{7}$, $M^{\text{free}} = \frac{6}{8}$).
- **GPU 2:** Occupied by one `2g.20gb` at index 2 and one `1g.10gb` at index 5 ($C^{\text{free}} = \frac{4}{7}$, $M^{\text{free}} = \frac{5}{8}$).

The paper explicitly reports the numerical fragmentation scores as:

$$F(1) = 8, \quad F(2) = 16$$

However, if one strictly evaluates all legal start indices ($\mathcal{S}_p$) for overlap (termed `all_legal_starts`), the calculated scores are:

$$F_{\text{all}}(1) = 14, \quad F_{\text{all}}(2) = 23$$

Profile	GPU 1 (Free)	GPU 1 (All)	GPU 2 (Free)	GPU 2 (All)
`7g.80gb`	0	0	0	0
`4g.40gb`	4	4	4	4
`3g.40gb`	4	4	8	8
`2g.20gb`	0	2	2	4
`1g.20gb`	0	2	2	4
`1g.10gb`	0	2	0	3
Total F	**8**	**14**	**16**	**23**

Table 2: Profile-by-Profile Breakdown Reconstructing MFI Figure 3 Scores [3].

As shown in Table 2, the discrepancy arises from how blocked configurations are counted. Under `all_legal_starts`, starts whose anchor index s is already occupied by a previously placed instance are counted as blocked. Under `free_anchor_starts`, a placement is only counted as blocked if its anchor index s is currently **free** ($u^{\text{comp}}[s] = 0 \wedge u^{\text{mem}}[s] = 0$), but its extended footprint is obstructed by a downstream occupied slice.

Filtering by free anchors yields exactly $F(1) = 8$ and $F(2) = 16$, perfectly resolving the numeric values published in Figure 3. We maintain both formulations in our research codebase to rigorously test sensitivity.

Paper-Path vs. Global-Feasible Baselines

Classical scheduling baselines in prior literature include Best-Fit Best-Index (BF-BI) and Worst-Fit Best-Index (WF-BI). In our audit, we identified two distinct operationalizations:

1. **Paper-Path Baselines (`paper_bf_bi`, `paper_wf_bi`):** The scheduler evaluates aggregate residual capacity across GPUs, selects the candidate GPU with minimum (BF) or maximum (WF) remaining resources, and then attempts placement using the Best-Index order. If the chosen GPU lacks a feasible contiguous placement, the job is **immediately rejected**, reflecting the exact rejection path described in [3].
2. **Global-Feasible Baselines:** The scheduler searches globally across all feasible (g, s) candidate pairs cluster-wide before ranking by residual capacity.

Paper-path baselines faithfully expose the spatial fragmentation vulnerabilities discussed in [3], whereas global-feasible variants mask them.

Bounded Lookahead & Candidate Pruning

Monte Carlo Rollout Formulation

To test whether anticipating future arrivals can improve placement decisions over greedy MFI, we formulate a bounded lookahead scheduler. Upon the arrival of job j, the scheduler identifies the set of all physically feasible candidate placements $\mathcal{A}(j) = \{(g, s)\}$.

For each candidate action $a \in \mathcal{A}(j)$, the scheduler simulates the allocation of a and executes R independent Monte Carlo rollouts over a finite horizon of $H = 2$ future job arrivals. During each rollout $r \in \{1, ..., R\}$, future workload arrivals and service durations are sampled from the observed online workload history. Future placement decisions inside the rollout are resolved using greedy MFI.

The empirical rollout value $\hat{V}_{H(a)}$ estimates the expected number of downstream jobs accepted:

$$\hat{V}_{H(a)} = \frac{1}{R} \sum_{r=1}^{R} Y_{r(a)}$$

where $Y_{r(a)}$ is the count of future jobs accepted in rollout r. To minimize comparison variance, we employ **Common Random Numbers (CRN)**, evaluating all competing candidates against identical sampled future traces.

Adaptive Stability & Ambiguity Gating

Evaluating R rollouts for every candidate on every arrival is computationally expensive. We designed an adaptive search policy governed by two principled gates:

1. **Forecast Stability Gate:** Using an additive-smoothed profile frequency estimator over recent arrivals $\hat{q}_{p(t)} = \frac{n_p + \eta}{N + K\eta}$, the scheduler computes the Total Variation (TV) distance between older and newer halves of the history window:

$$\mathrm{TV}(\hat{q}_{\mathrm{old}}, \hat{q}_{\mathrm{new}}) = \frac{1}{2} \sum_{p \in \mathcal{P}} |\hat{q}_{\mathrm{old},p} - \hat{q}_{\mathrm{new},p}|$$

A reliability score $R_t = \max\left(0, 1 - \frac{\mathrm{TV}}{\theta_{\mathrm{TV}}}\right)$ must exceed a minimum threshold ($\theta_{\mathrm{rel}} = 0.20$) to permit lookahead.
2. **Ambiguity Gate:** If MFI's top two candidates have a substantial fragmentation score gap ($M_t = \Delta F_{(2)} - \Delta F_{(1)} > \theta_{\mathrm{margin}}$), MFI is already decisive, and lookahead search is bypassed.
3. **Sequential Early Stopping:** Candidate evaluations proceed in batches of size 2. Paired Student-t confidence intervals are evaluated after each batch; search terminates early if the leading candidate statistically separates from all rivals.

GPU-Diverse Candidate Pruning

In a scaled cluster of 20 GPUs, a single request can yield 30 to 80 feasible (g, s) placements. Simulating rollouts across 80 candidates causes an immediate computational bottleneck.

In Milestone 14, we developed **GPU-Diverse Candidate Pruning**. Candidates are grouped by GPU, and only the single best placement per GPU (as ranked by MFI increment) is shortlisted. The top-ranked placements across distinct GPUs are retained up to a strict cap:

$$|\mathcal{A}_{\mathrm{pruned}}(j)| \leq K_{\max} = 5$$

This caps rollout evaluations to at most 5 per arrival while guaranteeing exploration across distinct physical GPUs.

Experimental Evaluation

Scaled Benchmark Configuration

We conduct a large-scale empirical evaluation comparing 10 scheduling policies across a 20-GPU cluster (140 compute slices, 160 memory slices). The benchmark comprises:

- **Cluster Scale:** 20 simulated NVIDIA A100-80GB GPUs.
- **Workload Distributions:** Four synthetic distributions from [3], Table II:
  - **Uniform:** equal probability across all 6 profiles ($\approx 16.7\%$ each).
  - **Skew-Small:** dominated by `1g.10gb` (40%) and `2g.20gb` (25%).
  - **Skew-Big:** dominated by `7g.80gb` (35%) and `4g.40gb` (25%).
  - **Bimodal:** concentrated on `1g.10gb` (40%) and `7g.80gb` (35%).
- **Arrival Regimes:**

  - **Progressive Load:** Target offered load 1.0 (cumulative paper slices equal to total cluster capacity), inter-arrival gap $\Delta t = 0.25$. Completed jobs continually free slices.
  - **Burst Load:** Target offered load 1.5, inter-arrival gap $\Delta t = 0.0$. All jobs arrive concurrently before any positive-duration job can complete.
- **Replication:** 20 independent seeds per distribution per regime, yielding 80 progressive traces and 80 burst traces (160 paired traces total; 1,600 full simulations).
- **Execution:** 16 parallel workers via Python `concurrent.futures`.

Regime	Policy	Mean Acc.	SD	Time (ms)	Compute	Memory
prog.	`paper_ff`	83.46%	9.11 pp	0.059	38.0%	39.6%
prog.	`paper_rr`	87.15%	5.04 pp	0.032	38.3%	39.6%
prog.	`paper_bf_bi`	86.86%	11.26 pp	0.113	40.9%	42.0%
prog.	`paper_wf_bi`	91.61%	4.03 pp	0.116	39.2%	40.2%
prog.	`mfi_all_starts`	99.91%	0.83 pp	0.662	43.1%	44.1%
prog.	`mfi_free_anchor`	99.79%	1.32 pp	0.691	43.0%	44.1%
prog.	Fixed Look. (all)	99.91%	0.83 pp	37.64	43.1%	44.1%
prog.	Fixed Look. (free)	99.79%	1.32 pp	36.51	43.0%	44.1%
prog.	Adapt. Look. (all)	99.91%	0.83 pp	25.55	43.1%	44.1%
prog.	Adapt. Look. (free)	99.79%	1.32 pp	38.69	43.0%	44.1%
burst	`paper_ff`	64.27%	6.10 pp	0.058	55.8%	58.4%
burst	`paper_rr`	71.87%	7.11 pp	0.060	48.3%	50.5%
burst	`paper_bf_bi`	66.14%	7.07 pp	0.101	60.2%	61.8%
burst	`paper_wf_bi`	76.51%	3.99 pp	0.166	53.2%	54.6%
burst	`mfi_all_starts`	71.97%	4.37 pp	0.545	62.0%	63.3%
burst	`mfi_free_anchor`	72.88%	4.10 pp	0.511	61.8%	63.2%
burst	Fixed Look. (all)	71.99%	4.38 pp	27.22	62.0%	63.3%
burst	Fixed Look. (free)	73.00%	4.00 pp	23.78	61.8%	63.2%
burst	Adapt. Look. (all)	71.96%	4.37 pp	17.42	62.0%	63.3%
burst	Adapt. Look. (free)	73.12%	3.82 pp	24.98	61.8%	63.2%

Table 3: Overall Workload Acceptance and Resource Utilization across 1,600 Simulations on a 20-GPU Cluster.

Diagnostic Offline Oracle Comparison

To establish theoretical upper bounds, we implemented an exact offline acceptance oracle via memoized dynamic programming over tiny traces (8 jobs, 1–2 GPUs) [5]. On 12 traces of 8 jobs on 1 GPU, mean online acceptance was 53.1%, with a mean oracle gap of 0.83 jobs. On 2 GPUs, mean online acceptance reached 77.3%, with an oracle gap of 0.55 jobs.

Crucially, dynamic programming state counts exploded combinatorially: a single 8-job trace on 2 GPUs required evaluating 234,000 memoized states and 10.3 seconds of CPU time. Expanding exact search beyond 10 jobs is intractable, cementing the necessity of heuristic evaluation.

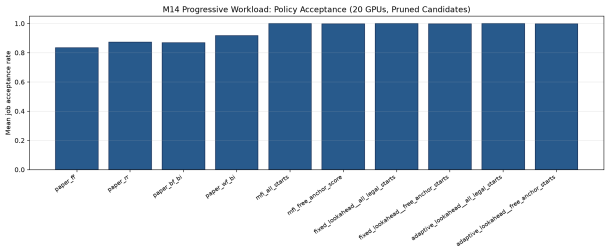


Figure 1: Workload Acceptance Comparison Under Progressive Load (20 GPUs, 80 Traces).

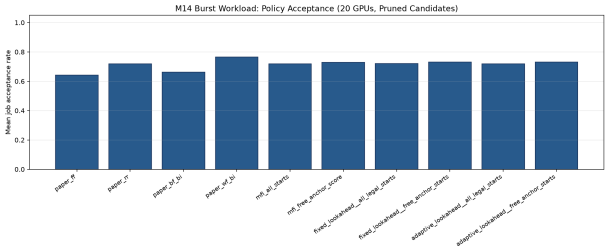


Figure 2: Workload Acceptance Comparison Under Burst Load (20 GPUs, 80 Traces).

Progressive-Load Acceptance Results

As shown in Table 3 and Figure 1, greedy MFI demonstrates near-optimal performance under progressive streaming load:

- `mfi_all_starts` achieves 99.91% acceptance.
- `mfi_free_anchor_score` achieves 99.79% acceptance.
- Both classical heuristics (`paper_ff` at 83.46%, `paper_rr` at 87.15%, `paper_bf_bi` at 86.86%) fall 12–16 percentage points behind MFI.

When evaluated against greedy MFI on identical paired traces:

- Fixed lookahead (`free_anchor_starts`) matches greedy MFI **identically** across all 80 traces: 0 wins, 0 losses, 80 ties (delta 0.00 pp, bootstrap 95% CI [0.00, 0.00]).
- Adaptive lookahead (`free_anchor_starts`) also matches greedy MFI identically (0 wins, 0 losses, 80 ties).
- Under `all_legal_starts`, lookahead records 3 wins, 0 losses, 77 ties (+0.12 pp, bootstrap 95% CI [0.00, 0.26] pp).

In summary, across 80 paired progressive traces, lookahead provides zero measurable advantage over myopic greedy MFI.

Regime	Policy vs. Ref.	Delta	Bootstrap 95% CI	W / L / T
prog.	`paper_ff`	−16.33 pp	[−18.35, −14.37]	0 / 79 / 1
prog.	`paper_rr`	−12.64 pp	[−13.70, −11.65]	0 / 80 / 0
prog.	`paper_bf_bi`	−12.93 pp	[−15.46, −10.46]	2 / 63 / 15
prog.	`paper_wf_bi`	−8.17 pp	[−9.10, −7.28]	0 / 79 / 1
prog.	`mfi_all_starts`	+0.12 pp	[+0.00, +0.27]	3 / 0 / 77
prog.	Fixed Look. (free)	+0.00 pp	[+0.00, +0.00]	0 / 0 / 80
prog.	Adapt. Look. (free)	+0.00 pp	[+0.00, +0.00]	0 / 0 / 80
burst	`paper_ff`	−8.61 pp	[−10.08, −7.17]	2 / 70 / 8
burst	`paper_rr`	−1.01 pp	[−2.81, +0.74]	32 / 43 / 5
burst	`paper_bf_bi`	−6.74 pp	[−8.16, −5.34]	7 / 62 / 11
burst	**`paper_wf_bi`**	**+3.63 pp**	[**+2.47, +4.79**]	**55 / 17 / 8**
burst	`mfi_all_starts`	−0.91 pp	[−1.40, −0.46]	14 / 42 / 24
burst	Fixed Look. (free)	+0.12 pp	[−0.05, +0.34]	5 / 3 / 72
burst	Adapt. Look. (free)	+0.24 pp	[+0.01, +0.54]	6 / 2 / 72

Table 4: Paired Acceptance Differences vs. Reference Free-Anchor MFI (10,000 Percentile Bootstrap 95% CIs).

Burst-Load Acceptance Results

Under the burst regime (Table 4 and Figure 2), cluster offered load reaches 150% with simultaneous arrivals:

- `mfi_free_anchor_score` achieves 72.88% acceptance.
- Fixed lookahead (`free_anchor_starts`) achieves 73.00% acceptance (+0.12 pp, bootstrap 95% CI [−0.05, +0.34] pp, spanning zero; 5 wins, 3 losses, 72 ties).
- Adaptive lookahead (`free_anchor_starts`) achieves 73.12% acceptance (+0.24 pp, bootstrap 95% CI [+0.01, +0.54] pp; 6 wins, 2 losses, 72 ties).

While adaptive lookahead records a marginal +0.24 pp difference, it is tied on 90% of traces. Most remarkably, Worst-Fit Best-Index (`paper_wf_bi`) achieves 76.51% acceptance, outperforming MFI by +3.63 pp (bootstrap 95% CI [+2.47, +4.79] pp, 55 wins, 17 losses, 8 ties).

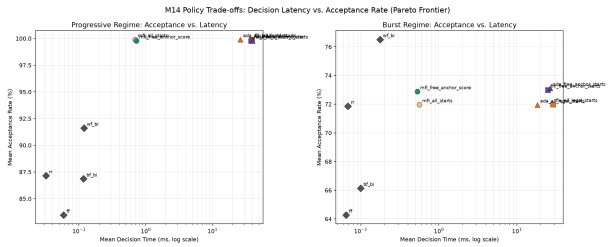


Figure 3: Decision Latency vs. Workload Acceptance Pareto Frontier (Log-Scaled Latency Axis).

Decision Latency Distribution

Table 5 and Figure 3 quantify the computational overhead of each policy:

- Classical baselines decide in 0.03–0.18 ms per request.
- Greedy MFI decides in 0.51–0.72 ms (median ≈ 0.23–0.41 ms).
- Lookahead policies require 24–40 ms on average—a 50× to 70× latency inflation.
- Tail latency is severe: P95 latency reaches 57–115 ms, P99 latency reaches 172–214 ms, and maximum decision times reach 952 ms.
- Total CPU time across all jobs in progressive load was 2.51 s for MFI vs. 132.0 s for fixed lookahead and 139.3 s for adaptive lookahead.

Regime	Policy	Mean	Median	P95	P99	Total CPU
prog.	`paper_ff`	0.059 ms	0.029 ms	0.050 ms	0.137 ms	0.20 s
prog.	`paper_rr`	0.032 ms	0.026 ms	0.045 ms	0.086 ms	0.11 s
prog.	`paper_bf_bi`	0.117 ms	0.080 ms	0.135 ms	0.334 ms	0.41 s
prog.	`paper_wf_bi`	0.119 ms	0.094 ms	0.147 ms	0.469 ms	0.41 s
prog.	`mfi_free_anchor`	0.721 ms	0.409 ms	1.464 ms	5.221 ms	2.51 s
prog.	Fixed Lookahead	37.98 ms	29.64 ms	98.54 ms	172.3 ms	132.0 s
prog.	Adapt. Lookahead	40.08 ms	34.94 ms	115.0 ms	213.6 ms	139.3 s
burst	`paper_ff`	0.065 ms	0.030 ms	0.058 ms	0.198 ms	0.34 s
burst	`paper_rr`	0.068 ms	0.028 ms	0.055 ms	0.219 ms	0.36 s
burst	`paper_bf_bi`	0.099 ms	0.066 ms	0.127 ms	0.402 ms	0.52 s
burst	`paper_wf_bi`	0.177 ms	0.087 ms	0.163 ms	0.718 ms	0.93 s
burst	`mfi_free_anchor`	0.529 ms	0.226 ms	1.356 ms	3.593 ms	2.78 s
burst	Fixed Lookahead	24.82 ms	23.28 ms	57.44 ms	173.7 ms	130.3 s
burst	Adapt. Lookahead	26.61 ms	23.61 ms	72.64 ms	187.6 ms	139.7 s

Table 5: Per-Decision Scheduling Latency Statistics and Total CPU Execution Time.

Score-Mode Sensitivity

We evaluated whether lookahead decisions differ when conditioned on `all_legal_starts` vs. `free_anchor_starts`:

- In progressive load, switching score modes changed 74.6% of per-job (g, s) placement tuples for both fixed and adaptive lookahead.
- In burst load, 52.8% of fixed lookahead and 52.9% of adaptive lookahead decisions changed.
- Despite altering 50%–75% of placement choices, overall acceptance shifted by only −0.12 pp in progressive load and +1.00 pp in burst load.

This confirms that placement choices are highly sensitive to the scoring formulation, but in streaming regimes, multiple alternative placements yield equivalent downstream capacity.

Discussion & Negative Results

Why Online Arrival Uncertainty Neutralizes Lookahead

A central theoretical question is: **Why did Monte Carlo lookahead fail to surpass greedy MFI?**

Our findings point to three fundamental factors:

1. **High Quality of the MFI Surrogate:** The MFI fragmentation score $F(g)$ is not a crude utilization proxy; it is a structural count of all blocked MIG configurations weighted by memory capacity. By greedily minimizing ΔF, MFI already preserves maximal future placement flexibility without needing to predict specific future arrival sequences.
2. **Arrival Horizon Truncation:** In an online system without job reservation queues, arrivals are non-deterministic. A shallow rollout horizon ($H = 2$) cannot anticipate arrival shifts occurring 5 to 10 steps ahead. Conversely, extending the horizon to $H \geq 5$ would cause exponential rollout path divergence, amplifying sampling noise.
3. **Duration and Departure Unpredictability:** In progressive streaming, job completions dynamically de-allocate slices, constantly reshaping physical contiguous regions. Simulating synthetic completions based on noisy lifetime estimates introduces errors that misguide rollout selection.

The Operational Cost of Rollout Latency

In production Kubernetes clusters managed by GPU device plugins, scheduling throughput is critical [1, 2]. Pod scheduling decisions must execute in sub-millisecond timeframes to prevent queue head-of-line blocking.

As demonstrated in Figure 3, greedy MFI resides on the Pareto frontier: it achieves 99.8% acceptance at 0.6 ms latency. Lookahead policies sit far off the Pareto frontier, imposing a 50×–70× latency tax for zero statistically reliable gain. Online rollout lookahead is therefore unviable for real-time cluster schedulers.

Why Spreading Beats Packing Under Burst Arrivals

The most impactful scientific insight of our 20-GPU study is that Worst-Fit Best-Index (`paper_wf_bi`) outperforms MFI by +3.63 pp under burst arrivals (76.51% vs. 72.88%).

The mechanism is rooted in spatial packing dynamics:

- **MFI is a Bin-Packing Strategy:** MFI seeks to minimize fragmentation increments by packing incoming jobs onto already active GPUs, preserving completely empty GPUs for future large allocations. Under streaming progressive load, this strategy is near-perfect (99.8% acceptance) because active jobs finish and release resources, maintaining clean boundaries.
- **Concurrent Burst Waves Invalidate Bin-Packing:** In a burst regime (inter-arrival gap 0.0), a large wave of jobs arrives simultaneously before any prior job can complete. MFI packs early smaller requests (e.g., `1g.10gb`, `2g.20gb`) onto the first few GPUs. This rapid spatial packing quickly eliminates the contiguous 4-slice and 7-slice anchor regions on those GPUs.
- **Worst-Fit Spreads Fragment Risk:** `paper_wf_bi` selects the GPU with maximum remaining residual resources, distributing early requests across all 20 GPUs. When com-

bined with Turkkan et al.'s Best Index ordering, each GPU receives at most one or two small instances placed at optimal fringe offsets (e.g., index 6 or 4). Consequently, **every GPU across the cluster retains a contiguous unallocated block**, allowing subsequent large requests (`3g.40gb`, `4g.40gb`, `7g.80gb`) to find feasible placements.

Conclusion & Future Work

In this paper, we systematically explored the performance and computational limits of online lookahead for fragmentation-aware MIG scheduling. Through an extensive empirical benchmark across 20 GPUs, 160 paired workload traces, and 1,600 simulations, we established that online Monte Carlo lookahead provides no statistically significant acceptance gain over greedy MFI, while introducing an intolerable 50×–70× latency penalty. We resolved the Figure 3 mathematical score ambiguity, proving that free-anchor filtering reproduces published values. Finally, we discovered that under zero-gap burst arrivals, load-spreading heuristics outperform greedy packing by +3.63 pp.

Based on these findings, we recommend:

1. **Production Deployment:** Maintain greedy MFI (`mfi_free_anchor_score`) as the primary online heuristic for streaming cloud workloads.
2. **Rejection of Lookahead:** Abandon online rollout search in production schedulers due to negative operational ROI.
3. **Regime-Aware Hybrid Schedulers:** Future systems should implement dynamic regime detection, switching from MFI during streaming traffic to Worst-Fit Best-Index during concurrent burst arrivals.

References

[1] NVIDIA Corporation, "NVIDIA Multi-Instance GPU User Guide," Santa Clara, CA, Tech. Rep., 2024. [Online]. Available: https://docs.nvidia.com/datacenter/tesla/mig-user-guide/

[2] NVIDIA Corporation, "NVIDIA A100 Tensor Core GPU Architecture Whitepaper," Santa Clara, CA, Tech. Rep., 2020.

[3] A. Zambianco, M. Fasol, and R. Doriguzzi-Corin, "An Online Fragmentation-Aware GPU Scheduler for Multi-Tenant MIG-based Clouds," **arXiv preprint arXiv:2511.18906**, 2024.

[4] E. Turkkan, O. Boyar, and B. Celik, "Optimal Workload Placement on Multi-Instance GPUs," **arXiv preprint arXiv:2409.06646**, 2024.

[5] S. Kumar, "MIG Scheduling Research Repository: Exact Offline Acceptance Oracle & Lookahead Benchmark," 2026. [Online]. Available: https://github.com/saurabh-sah/mig_scheduling